



DWH AI-powered development: repo

15/04/2026

Holland & Barrett



Repo model organization



The repo is the single source of truth for DWH metadata

All metadata in and around the warehouse — model metadata, documentation, ownership, SLAs, tests, and runbooks — is curated in the repo and published outward from there.



Keep truth close to the model

The files that define, describe, and operate a model live in the model's own folder.



Structure for both humans and machines

Use predictable filenames and a clear metadata contract so engineers, automation, and AI can all work with the same pattern.



Capture judgment explicitly, generate the rest

Owners, SLA, criticality, purpose, and runbooks are curated; lineage, catalog views, and rendered docs are generated.



One source, many views

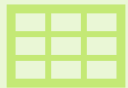
The repo is the source of truth, while Confluence, catalog pages, and management lists are published views of that source.

Repo model organization



Data model

A single technical data asset, usually one table or view, created for a specific purpose and owned at table level. It has defined logic, grain, metadata, documentation, tests, and operational procedures.



Data product

A business-facing data deliverable made up of one or more related data models, often across multiple layers, designed to support a specific use case or capability. It has clear ownership, canonical outputs, an end-to-end SLA, and consumer guidance.



*Data domain

A higher-level business area that groups related data products under a common context, ownership, and governance model. It helps organize, navigate, and manage the data landscape for a function such as finance, procurement, or customer.

Data model

Repo data model (single)

/layers/adl/finance/fact/fact_supplier_payment_terms_compliance/
fact_supplier_payment_terms_compliance.sql
model.yml
README.md
runbook.md
tests.yml
/derived

<model>.sql

- transformation logic

model.yml

- structured metadata

README.md

- business + technical overview

runbook.md

- ops / recovery guide

tests.yml

- declared quality checks

model.yml reference

```
model:
  name: # Exact technical name of the model or table.
  domain: # Business domain the model belongs to.
  schema: # DWH Schema where table is placed
  layer: # Architectural layer where the model sits, such as odl, adl

owners:
  workstream: # Workstream where this table is being developed
  analytics_owner: # Person or team responsible for analytical correctness and usage.
  business_owner: # Business owner accountable for business meaning and fit-for-purpose use.
  technical_owner: # Engineering owner responsible for implementation and support.

service:
  sla: # Expected availability target for the model.
  refresh_cadence: # Expected refresh frequency.
  criticality: # Business and operational importance of the model.

description:
  short: # One-line summary of what the model is.
  business_purpose: # Short explanation of why the model exists and what business need it supports.

documentation:
  main: # Path to the main human-readable model documentation file.
  runbook: # Path to the operational support and recovery guide.

classification:
  canonical: # Whether this is a preferred business consumption object. Some models are not intended for direct consumption.
  contains_pii: # Whether the model contains personal data.
  confidentiality: # Sensitivity classification of the model.

tags:
  - # Search and grouping labels for the model.
```

model.yml example

```
model:
  name: fact_supplier_payment_terms_compliance
  domain: finance
  schema: finance
  layer: adl

owners:
  workstream: Finance
  analytics_owner: <bi team/developer email>
  business_owner: <decision maker email>
  technical_owner: <dwh developer email>

service:
  sla: "Daily by 07:00 Europe/Amsterdam"
  refresh_cadence: daily
  criticality: high

description:
  short: "Fact model measuring supplier payment terms compliance."
  business_purpose: "Tracks compliance of supplier payments against agreed payment terms"

documentation:
  main: "./README.md"
  runbook: "./runbook.md"

classification:
  canonical: true
  contains_pii: false
  confidentiality: internal

tags:
  - procurement
  - finance
  - compliance
```

README.md reference

```
# <!-- Title: Exact model name. -->

## Purpose
<!-- Purpose: Short explanation of what the model is for and why it exists. -->

## Grain
<!-- Grain: Definition of what a single row represents. -->

## Table structure

| Column | Type | Description |
|---|---|---|
| | | |
| | | |
| | | |
<!-- Table structure: Most important columns of the model with their data types and business meaning. -->

## Key business rules
<!-- Key business rules: Main logic or interpretation rules users should know. -->

## Upstream inputs
<!-- Upstream inputs: Main source tables or models used to build this model. -->

## Downstream usage
<!-- Downstream usage: Main reports, dashboards, teams, or use cases that consume this model. -->

## SLA
<!-- SLA: Expected availability target for the model. -->

## Criticality
<!-- Criticality: Business and operational importance of the model. -->
```

README.md example

```
# fact_supplier_payment_terms_compliance

## Purpose
Tracks supplier invoice payment terms compliance for finance and procurement analysis.

## Grain
One row per invoice (`invoice_sk`), keeping the latest available record after deduplication.

## Table structure
| Column | Type | Description |
|---|---|---|
| invoice_sk | varchar(200) | Surrogate key of the invoice, built from source and invoice number. |
| supplier_invoice_nk | varchar(100) | Supplier-facing invoice number from Unit4. |
| supplier_sk | varchar(200) | Standardized supplier key linked to the supplier dimension. |
| date_invoice_nk | date | Invoice date. |
| date_invoice_due_nk | date | Due date of the invoice based on payment terms. |
| date_invoice_paid_nk | date | Actual payment date of the invoice. |
| remaining_amount_eur | numeric(18,2) | Remaining unpaid invoice amount converted to EUR. |

## Key business rules
- Input is sourced from Unit4 seed and history tables.
- Records are deduplicated by `invoice_sk`, keeping the latest version by `ingest_date` and `date_updated_nk`.
- Remaining and currency amounts are enriched with FX rates to produce EUR and GBP values.

## Upstream inputs
- `rdl_unit4.payment_term_compliance_seed`
- `rdl_unit4.payment_term_compliance_history`
- `odl.dim_suppliers`

## Downstream usage- supplier payment monitoring
- finance reporting

## SLA
Daily by 07:00 Europe/Amsterdam
```

runbook.md reference

```
# Runbook: fact_trading_performance_daily_email
<!-- Title: Model name prefixed with "Runbook:". -->

## What this model supports
<!-- What this model supports: Short explanation of the business process, reporting, or use case supported by the model. -->

## Expected refresh
<!-- Expected refresh: Target refresh timing or availability expectation for the model. -->

## Failure impact
<!-- Failure impact: What is affected if the model is late, wrong, or unavailable. -->

## Common failure modes
<!-- Common failure modes: Most likely reasons the model could fail or produce incorrect results. -->

## Diagnostics
<!-- Diagnostics: Ordered checks to identify the source of the issue. -->

## Recovery steps
<!-- Recovery steps: Ordered actions to restore the model and validate the fix. -->

## Escalation
<!-- Escalation: People or teams to contact when recovery is not straightforward or SLA risk is high. -->

## Related checks
<!-- Related checks: Key validations to confirm the model is healthy after recovery. -->
```

runbook.md example

Please visit <https://hbidigital.atlassian.net/wiki/x/UAAUFAI>

tests.yml reference

duplicate_tests:

- name: # Name of the duplicate test.
- description: # Short explanation of what duplicate condition is being checked.
- enabled: # Whether the test is active and should be executed.
- schedule: # When the test should run, for example with the model, daily, hourly, weekly, or ad_hoc.
- field: [] # Column or columns that should be unique at the tested grain.
- grouping: [] # Optional grouping expressions used to test uniqueness within a partition or business slice.
- filter: [] # Optional filter conditions limiting which records are included in the test.
- severity: # Importance of the test result, such as warning or error.

gap_tests:

- name: # Name of the gap test.
- description: # Short explanation of what missing data, missing dates, or missing sequence is being checked.
- enabled: # Whether the test is active and should be executed.
- schedule: # When the test should run, for example with the model, daily, hourly, weekly, or ad_hoc.
- field: [] # Column or columns the gap test is focused on.
- grouping: [] # Optional grouping expressions used to assess completeness within a partition or business slice.
- filter: [] # Optional filter conditions limiting which records are included in the test.
- skip_dates: [] # Specific dates that should be excluded from the gap check, for example planned non-business days or known source outages.
- severity: # Importance of the test result, such as warning or error.

discrepancy_tests:

- name: # Name of the discrepancy test.
- description: # Short explanation of what mismatch or reconciliation is being checked.
- enabled: # Whether the test is active and should be executed.
- schedule: # When the test should run, for example with the model, daily, hourly, weekly, or ad_hoc.
- threshold: # Allowed level of difference before the test fails, for example 0, 0.01, 1 percent, or an agreed business tolerance.
- query: # Path or link to the SQL query implementing the discrepancy check.
- severity: # Importance of the test result, such as warning or error.

tests.yml example

```
duplicate_tests:
- name: invoice_sk
  description: Ensure one latest record exists per invoice.
  enabled: true
  schedule: with_model
  field:
    - invoice_sk
  grouping:
    - DATE_TRUNC('month', date_invoice_nk)
  filter:
    - 1=1
  severity: error

gap_tests:
- name: missing_invoice_date
  description: Ensure invoice date is populated for all records.
  enabled: true
  schedule: daily
  field:
    - date_invoice_nk
  grouping:
    - DATE_TRUNC('month', ingest_date)
  filter:
    - 1=1
  skip_dates:
    - '2026-01-01'
    - '2026-12-25'
  severity: error

discrepancy_tests:
- name: remaining_amount_eur_reconciliation
  description: Ensure EUR remaining amount reconciles within agreed tolerance.
  enabled: false
  schedule: weekly
  threshold: 0.01
  query: ./checks/payment_terms_reconciliation.sql
  severity: error
```

Data product

Repo data model (single)

/data_products/finance/availability/
product.yml
README.md
runbook.md

product.yml

- structured metadata

README.md

- business + technical overview

runbook.md

- ops / recovery guide

tests.yml

- declared quality checks

product.yml reference

```
product:
  name: # Exact name of the data product.
  domain: # Business domain the data product belongs to.

owners:
  workstream: # Workstream where this table is being developed
  analytics_owner: # Person or team responsible for analytical correctness and adoption.
  business_owner: # Business owner accountable for the product's meaning and value.
  technical_owner: # Engineering owner responsible for implementation and support.

service:
  sla: # End-to-end availability target for the product.
  refresh_cadence: # Expected refresh frequency.
  criticality: # Business and operational importance of the product.

description:
  short: # One-line summary of the data product.
  business_purpose: # Explanation of why the product exists and what business need it supports.

outputs:
  canonical_models: # Final models that consumers should use by default.

components:
  models: # Models that belong to the data product across all layers.

composition:
  depends_on_products: # Other data products this product relies on.

documentation:
  main: # Path to the main product documentation file.
  runbook: # Path to the end-to-end operational support guide.

classification:
  certified_for_reporting: # Whether the data product is approved for reporting use.
  confidentiality: # Sensitivity classification of the product.

tags:
  - # Search and grouping labels for the product.
```

product.yml example

```
product:
  name: supplier_payment_terms_compliance
  domain: finance

owners:
  workstream: finance
  analytics_owner: data.team@company.com
  business_owner: procurement.ops@company.com
  technical_owner: dwh.team@company.com

service:
  sla: "Daily by 08:00 Europe/Amsterdam"
  refresh_cadence: daily
  criticality: high

description:
  short: "Data product for supplier payment terms compliance reporting."
  business_purpose: "Provides the end-to-end data foundation for monitoring supplier payment terms compliance."

outputs:
  canonical_models:
    - fact_supplier_payment_terms_compliance

components:
  models:
    - odl_supplier_invoice_events
    - int_supplier_payment_terms
    - fact_supplier_payment_terms_compliance

composition:
  depends_on_products:
    - supplier_master

documentation:
  main: "./README.md"
  runbook: "./runbook.md"

classification:
  certified_for_reporting: true
  confidentiality: internal

tags:
  - procurement
  - finance
  - compliance
```

README.md reference

```
#  
<!-- Title: Exact data product name. -->  
  
## Purpose  
<!-- Purpose: Short explanation of what the data product is for and why it exists. -->  
  
## Scope  
<!-- Scope: Description of what layers, processes, or model groups are included in the product. -->  
  
## Canonical outputs  
-  
<!-- Canonical outputs: Final models that consumers should use by default. -->  
  
## Included models  
-  
<!-- Included models: Main models that belong to the data product across layers. -->  
  
## Depends on products  
-  
<!-- Depends on products: Other data products this product relies on. -->  
  
## Consumer guidance  
<!-- Consumer guidance: Explanation of which models are intended for broad use and how the product should be consumed. -->  
  
## SLA  
<!-- SLA: End-to-end availability target for the data product. -->  
  
## Criticality  
<!-- Criticality: Business and operational importance of the data product. -->
```

README.md example

```
# supplier_payment_terms_compliance
```

```
## Purpose
```

```
Provides the end-to-end data foundation for monitoring supplier payment terms compliance.
```

```
## Scope
```

```
Covers operational inputs, reusable transformations, and final mart outputs used in finance reporting.
```

```
## Canonical outputs
```

```
- fact_supplier_payment_terms_compliance
```

```
## Included models
```

```
- odl_supplier_invoice_events
```

```
- int_supplier_payment_terms
```

```
- fact_supplier_payment_terms_compliance
```

```
## Depends on products
```

```
- supplier_master
```

```
- fx_rates
```

```
## Consumer guidance
```

```
Business users and analysts should consume the canonical output model unless a reusable component is explicitly required.
```

```
## SLA
```

```
Daily by 08:00 Europe/Amsterdam
```

```
## Criticality
```

```
High
```

runbook.md reference

```
# Runbook:
<!-- Title: Data product name prefixed with "Runbook:". -->

## Overview
<!-- Overview: Short description of what the data product supports. -->

## Dependencies
-
<!-- Dependencies: Main upstream systems, shared components, or services required by the product. -->

## Expected refresh
- Cadence:
- SLA:
<!-- Expected refresh: Planned refresh frequency and expected delivery time. -->

## Failure impact
<!-- Failure impact: What business processes, reports, or users are affected if the product is wrong, late, or unavailable. -->

## Common failure modes
<!-- Common failure modes: Most likely technical or data issues affecting the product end-to-end. -->

## Diagnostics
<!-- Diagnostics: Ordered checks used to identify the source of the issue across the product. -->

## Recovery steps
<!-- Recovery steps: Ordered actions to restore the data product and verify recovery. -->
```

tests.yml reference

completeness_tests:

- name: # Name of the completeness test.
- description: # Short explanation of what required coverage, presence, or availability is being checked.
- enabled: # Whether the test is active and should be executed.
- schedule: # When the test should run, for example with _product, daily, weekly, monthly, or ad_hoc.
- field: [] # Column or columns the completeness test is focused on.
- grouping: [] # Optional grouping expressions used to assess completeness within a partition or business slice.
- filter: [] # Optional filter conditions limiting which records are included in the test.
- skip_dates: [] # Specific dates that should be excluded from the completeness check.
- severity: # Importance of the test result, such as warning or error.

consistency_tests:

- name: # Name of the consistency test.
- description: # Short explanation of what cross-model or cross-output consistency is being checked.
- enabled: # Whether the test is active and should be executed.
- schedule: # When the test should run, for example with _product, daily, weekly, monthly, or ad_hoc.
- threshold: # Allowed level of difference before the test fails.
- query: # Path or link to the SQL query implementing the consistency test.
- severity: # Importance of the test result, such as warning or error.

tests.yml example

```
completeness_tests:
- name: latest_business_date_present
  description: Ensure the canonical output contains the latest expected business date.
  enabled: true
  schedule: daily
  field:
    - date_invoice_nk
  grouping:
    - country_sk
  filter:
    - 1=1
  skip_dates:
    - '2026-01-01'
    - '2026-12-25'
  severity: error

consistency_tests:
- name: reusable_component_vs_canonical_output
  description: Ensure key aggregates remain consistent between shared intermediate logic and canonical output.
  enabled: true
  schedule: weekly
  threshold: 0.01
  query: ./checks/reusable_component_vs_canonical_output.sql
  severity: error
```

Data domain

Repo data model (single)

/domains/finance/
domain.yml
README.md

domain.yml

- structured metadata

README.md

- business + technical overview

domain.yml reference

```
domain:  
  name: # Exact name of the domain.  
  
owners:  
  analytics_owner: # Person or team responsible for analytical correctness and adoption.  
  business_owner: # Business owner accountable for the domain's meaning and value.  
  technical_owner: # Engineering owner responsible for implementation and support.  
  
description:  
  short: # One-line summary of the domain.  
  business_purpose: # Explanation of why the domain exists and what business area it supports.  
  
products:  
  included: # Data products that belong to this domain.  
  canonical: # Preferred or business-facing data products in this domain.  
  
classification:  
  confidentiality: # Sensitivity classification of the domain.  
  certified_for_reporting: # Whether the domain includes reporting-certified products.  
  
documentation:  
  main: # Path to the main domain documentation file.  
  
tags:  
  - # Search and grouping labels for the domain.
```

domain.yml example

```
domain:
  name: finance
  status: active
  lifecycle_stage: production

owners:
  analytics_owner: data.team@company.com
  business_owner: finance.ops@company.com
  technical_owner: dwh.team@company.com

description:
  short: "Domain covering finance data products and reporting."
  business_purpose: "Provides trusted data products for finance operations, performance analysis, and reporting."

products:
  included:
    - supplier_payment_terms_compliance
    - accounts_payable
    - working_capital

  canonical:
    - supplier_payment_terms_compliance
    - working_capital

classification:
  confidentiality: internal
  certified_for_reporting: true

documentation:
  main: "./README.md"

tags:
  - finance
  - accounts_payable
  - working_capital
  - compliance
```

README.md reference

```
#
<!-- Title: Exact domain name. -->

## Purpose
<!-- Purpose: Short explanation of what the domain is for and why it exists. -->

## Scope
<!-- Scope: Description of the business area and major product groups covered by the domain. -->

## Included data products
-
<!-- Included data products: Main data products belonging to the domain. -->

## Canonical data products
-
<!-- Canonical data products: Preferred or business-facing products in the domain. -->

## Consumer guidance
<!-- Consumer guidance: Explanation of how users should navigate or consume products within the domain. -->

## Ownership
- Analytics owner:
- Business owner:
- Technical owner:
<!-- Ownership: Domain-level accountability for analytics, business, and engineering. -->
```

README.md example

```
# finance

## Purpose
Provides trusted data products for finance operations, performance analysis, and reporting.

## Scope
This domain covers finance-related data products, including accounts payable, working capital, and supplier payment terms compliance.

## Included data products
- supplier_payment_terms_compliance
- accounts_payable
- working_capital

## Canonical data products
- supplier_payment_terms_compliance
- working_capital

## Consumer guidance
Users should start from canonical data products unless a more specific product is required.

## Ownership
- Analytics owner: data.team@company.com
- Business owner: finance.ops@company.com
- Technical owner: dwh.team@company.com
```

Thank you

Holland & Barrett